



ALUMNO:
Itzamara Elizabeth Rodriguez Ruiz.

ID:
00000281866.

ASIGNACIÓN:
Investigación OutputStream/InputStream y los BufferedReader/BufferedWriter en JAVA.

FECHA:
11 de Septiembre de 2026.

MATERIA:
Programación lll.

PROFESOR:
José Luis Robles Reyes.

De manera individual, investigar para que se utilizan los OutputStream/InputStream y los BufferedReader/BufferedWriter en JAVA, y traer un ejemplo básico.

Uso de OutputStream/InputStream vs. BufferedReader/BufferedWriter

InputStream y OutputStream

Trabajan con **datos binarios puros (bytes)** sin interpretar el tipo de codificación de texto.

- **Uso principal:** Lectura y escritura de archivos binarios como imágenes, videos, audios, ejecutables, archivos comprimidos (.zip), o transacciones puras por sockets de red.
- **Clases comunes:** FileInputStream, FileOutputStream.

BufferedReader y BufferedWriter

Son decoradores que trabajan con **texto (caracteres de 16 bits)** y utilizan un **buffer de memoria intermedia** (generalmente de 8 KB por defecto) para minimizar los accesos directos al disco o a la red.

Uso principal: Lectura y escritura eficiente de archivos de texto plano (.txt, .csv, .json, .xml, archivos de configuración, etc.) de forma orientada a líneas.

- Cuándo conviene usar Streams y cuando conviene usar Buffered

Según el tipo de contenido:

- **Streams (InputStream/OutputStream):** Conviene usarlos para archivos binarios (imágenes, audios, PDFs, ejecutables).
- **Buffered (BufferedReader/BufferedWriter):** Conviene usarlos para texto plano (.txt, .log, .csv, .json).

Según el volumen y las operaciones:

- **Streams (InputStream/OutputStream):** Conviene usarlos en lecturas o escrituras pequeñas, o en flujos directos donde la codificación de texto pueda destruir los datos.
- **Buffered (BufferedReader/BufferedWriter):** Conviene usarlos en lecturas o escrituras masivas o frecuentes de texto donde el acceso directo a disco sea el cuello de botella.

Según la lectura por estructura:

- **Streams (InputStream/OutputStream):** Conviene usarlos para acceso byte a byte o en bloques de bytes arbitrarios.
- **Buffered (BufferedReader/BufferedWriter):** Conviene usarlos cuando existe la necesidad de procesar el archivo línea por línea (usando el método readLine()).

- ¿Qué consideraciones hay que tener?

  Gestión de recursos (Cierre de Flujos): Siempre debes cerrar los flujos para evitar *resource leaks* (fugas de memoria o bloqueo de archivos en el sistema operativo). Es preferible utilizar **try-with-resources** (como en los ejemplos superiores).

  Uso de flush(): Al escribir con un buffer (**BufferedWriter**), los datos permanecen en RAM hasta que el buffer se llena. Si la aplicación termina inesperadamente antes de llenar el buffer, los datos no se guardan en el disco. **bw.flush()** fuerza la escritura inmediata a disco.

  Codificación de caracteres (Encoding): FileReader y **FileWriter** usan la codificación por defecto del sistema. Para asegurar UTF-8 u otro estándar, envuelve el Stream con **InputStreamReader/OutputStreamWriter:**

  Impacto en Memoria: No almacenes archivos enteros en memoria dentro de colecciones si solo necesitas procesarlos línea por línea; aprovecha la lectura en streaming con **readLine().**

- ¿Cuál es su sintaxis?

```
// Para texto (Reader / Writer)

BufferedReader br = new BufferedReader(new FileReader("archivo.txt"));

BufferedWriter bw = new BufferedWriter(new FileWriter("archivo.txt"));

// Para texto especificando tamaño de buffer personalizado (ej. 16 KB)

BufferedReader brCustom = new BufferedReader(new
FileReader("archivo.txt"), 16384);

// Para binarios (Stream)

BufferedInputStream bis = new BufferedInputStream(new
FileInputStream("imagen.png"));

BufferedOutputStream bos = new BufferedOutputStream(new
FileOutputStream("imagen.png"));
```

- ¿Qué restricciones tiene?

  InputStream/OutputStream no manejan codificación de caracteres: Si intentas leer texto multi-byte (como UTF-8 con tildes o caracteres especiales) byte por byte, romperás los caracteres al no procesar los bytes agrupados correspondientes.

  Sin acceso aleatorio: Ambos modelos son flujos secuenciales. No puedes "saltar" arbitrariamente a una posición específica del archivo para modificarla sin reescribir (para acceso aleatorio se usa RandomAccessFile o las APIs de java.nio).

  Operaciones bloqueantes: Las operaciones de lectura/escritura convencionales de java.io bloquean el hilo (*thread*) de ejecución hasta que la operación I/O finalice.

readLine() ignora saltos de línea: El método readLine() de BufferedReader devuelve el texto omitiendo los caracteres \n o \r\n, por lo que debes agregarlos explícitamente (newLine()) si estás reescribiendo.